

ASSIGNMENT: Module 3 – Lab: Implement VeraCrypt

1. Security Scenario and Plausible Deniability

While corporate espionage and unlawful access by malicious actors are possible scenarios, most of the cases I could find on the use of hidden volumes seemed to indicate that it's used by people who are up to no good anyway – such as the police officer picked upon suspicion of having child pornography, or a Brazilian banker absconding with secret information. A quick search on google indicates that this is also used by journalists to protect sources and other sensitive information, and allegedly this type of method was utilized by Edward Snowden. Honestly, there is no information that I ever had access to at any of my former employers that would be worth the effort of encryption or giving up my liberty/life.

That said, it's good to have options if you have data that would have real world consequences in the hands of the wrong people. If I were attached to the Foreign Service of the US State Department, I'm pretty sure that they have sophisticated protocols and requirements for Foreign Service Officers (FSO) who are traveling and working out of the US.

2. Hidden Volume Research & Concept Explanation

VeraCrypt's hidden volume feature directly addresses this type of coercive threat. A hidden volume allows the user to create two encrypted environments inside a single container: an outer “decoy” volume with harmless files and a second, concealed volume that contains the sensitive data. If a traveler is pressured to reveal a password, they can safely disclose the outer volume password without exposing the hidden one. Because the hidden volume is indistinguishable from random data and has no identifiable signature, it provides true plausible deniability during a forced search.

VeraCrypt implements hidden volumes using a dual-layer encryption design. The outer volume behaves like a normal encrypted container, while the hidden volume resides in the unused space of that container and can only be accessed with a separate password. Both layers use strong cryptographic algorithms, and the hidden volume's structure is intentionally indistinguishable from random noise. This means that even forensic tools cannot determine whether a hidden volume exists. This architecture allows a user to plausibly deny the existence of the sensitive data while still complying with a demand to unlock the outer volume.

This lab required installing and configuring VeraCrypt inside a Kali Linux virtual machine. Although the steps themselves were straightforward, the overall process took more than four hours due to multiple technical issues with VirtualBox, display scaling, and file handling.

3. Hidden Volume Implementation Documentation

The first step was preparing the environment. I switched VirtualBox from Basic Mode to Expert Mode to access the full VM configuration options (**Screenshot 1**). I then cloned my existing Kali VM to create a dedicated lab4 instance (**Screenshot 2**). After the clone completed, I logged into the new VM (**Screenshot 3**) and verified the system architecture using `uname -m` (**Screenshot 4**) to ensure I downloaded the correct VeraCrypt AppImage.

Once VeraCrypt was downloaded and made executable, I launched the application and opened the Volume Creation Wizard (**Screenshot 5**). For this assignment, I needed to demonstrate plausible deniability, so I selected the Hidden Volume option (**Screenshot 6**). I then chose a location for the encrypted container on the Desktop (**Screenshot 7**) and specified the volume size (**Screenshot 8**).

VeraCrypt requires either a password or a keyfile. For stronger security, I generated a random keyfile by moving the mouse to increase entropy (**Screenshot 9**). After the keyfile was created, I

completed the outer volume setup and received confirmation that the volume was successfully created (**Screenshot 10**).

Next, I mounted the encrypted container using the keyfile (**Screenshot 11**). Once mounted, I navigated to the mount directory in the terminal and created a test file inside the encrypted volume using nano (**Screenshot 12**). This confirmed that the volume was functioning correctly and that data written inside the mount point was encrypted on disk.

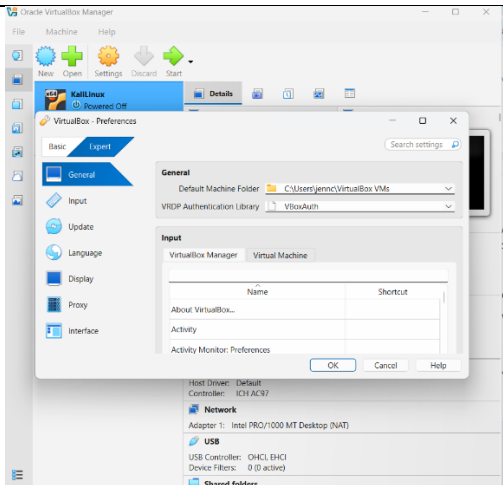
Then, while I was reviewing the screen captures and putting them into this Word document, the lab4 VM froze and I spent the better part of 30 minutes trying to get back into it to take additional screen captures. Next, I accidentally closed the Word document without saving, which required me to recreate the entire table and reinsert the screenshots. Overall, this lab took approximately four hours to complete, primarily due to VM instability, memory issues, and VirtualBox freezing. It was a learning experience, but more of the kind that results in aversion rather than enthusiasm for another round with VeraCrypt and Kali Linux.

CONCLUSION: This lab demonstrated how VeraCrypt's hidden volume feature can be used to protect sensitive data in high-risk travel scenarios where coercion or device inspection is possible. By creating both an outer decoy volume and a concealed hidden volume, a user can comply with a demand to reveal a password without exposing the truly sensitive information. Although the technical steps were manageable, the process highlighted the practical challenges of working within virtualized environments, including display issues, VM instability, and file-handling problems. Despite these obstacles, the final implementation successfully produced a functional hidden volume and validated the core concept of plausible deniability. This exercise reinforced the importance of layered encryption strategies and the operational realities of maintaining secure data practices in the field.

APPENDIX: Screen Captures

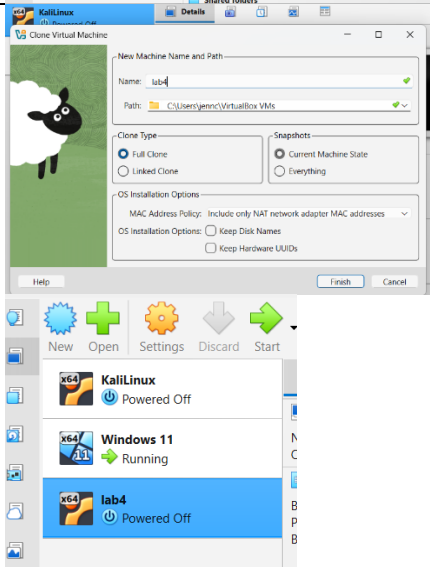
Screenshot 1

VirtualBox Preferences — switching from Basic to Expert Mode.



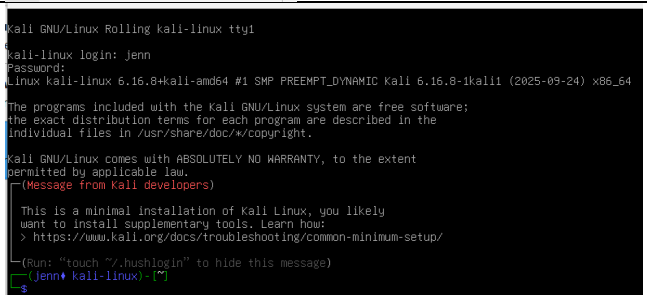
Screenshot 2

Cloning the Kali VM (lab4 creation window).



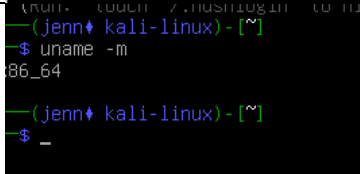
Screenshot 3

Kali Linux login screen for the lab4 VM.



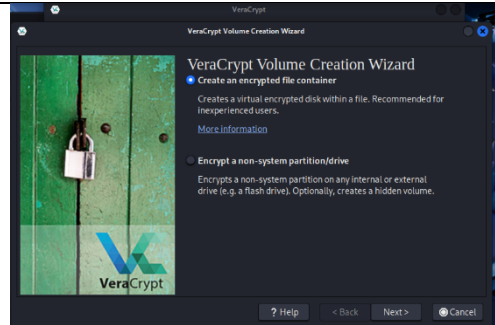
Screenshot 4

Terminal showing uname -m output (x86_64).



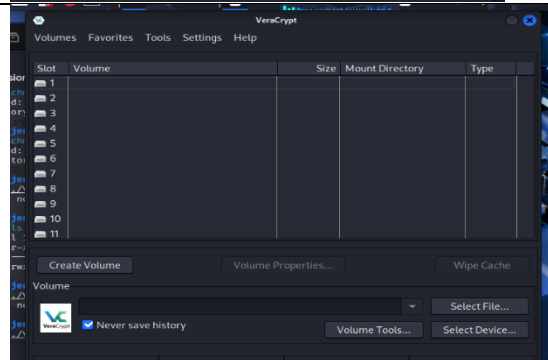
Screenshot 5

VeraCrypt Volume Creation Wizard —
“Create an encrypted file container.”



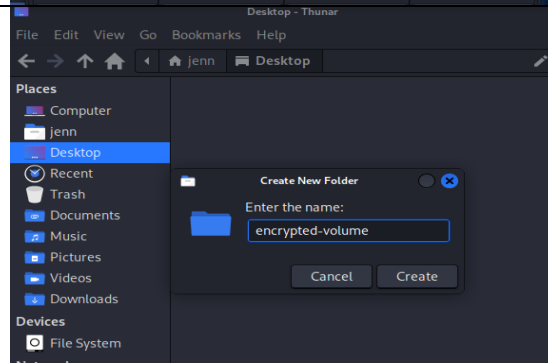
Screenshot 6

VeraCrypt — selecting “Hidden
VeraCrypt Volume.”



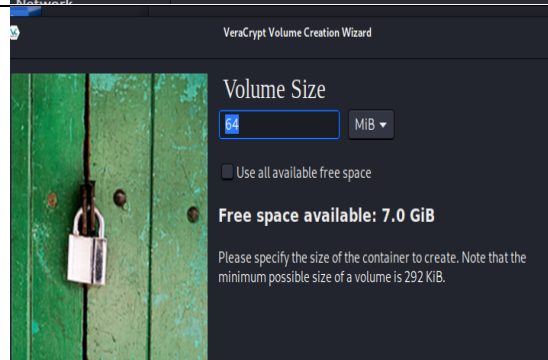
Screenshot 7

File selection dialog — choosing Desktop
and naming encrypted-volume.



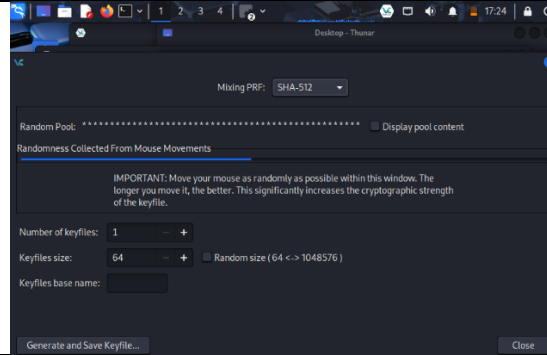
Screenshot 8

Volume Size screen — selecting 64 MiB.



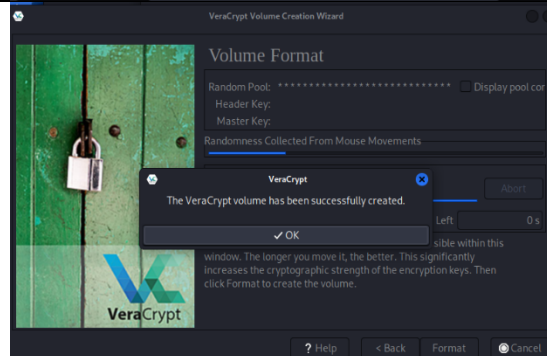
Screenshot 9

Keyfile generation — mouse-movement entropy window.



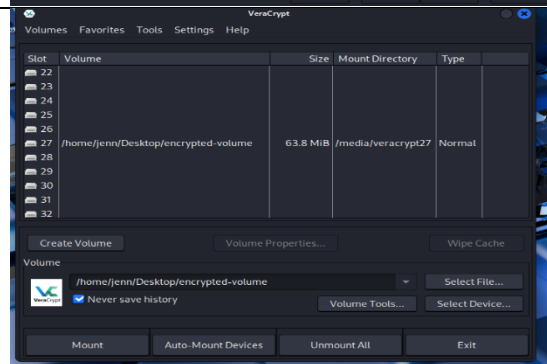
Screenshot 10

Confirmation — “The VeraCrypt volume has been successfully created.”



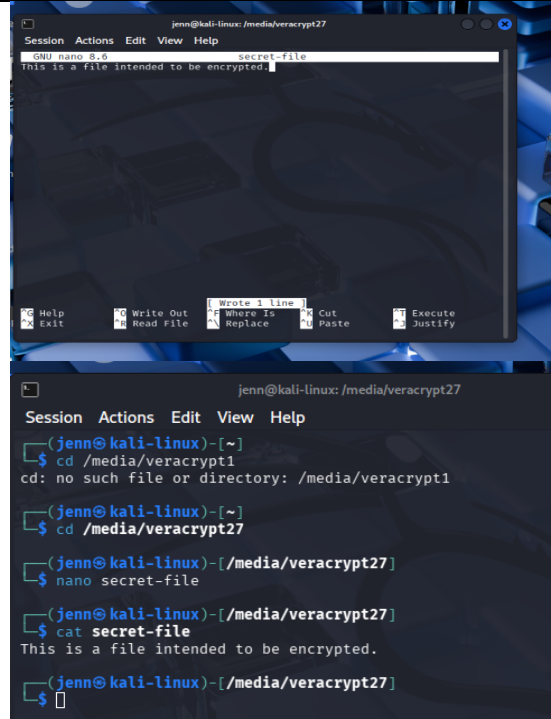
Screenshot 11

Mounting the volume — selecting the file and entering password/keyfile.



Screenshot 12

Terminal — creating secret-file inside /media/veracryptXX using nano.



The screenshot displays a terminal window on a Kali Linux system. The window title is 'jenn@kali-linux: /media/veracrypt27'. The terminal shows the following sequence of commands and output:

```
jenn@kali-linux: /media/veracrypt27
Session Actions Edit View Help
GNU nano 8.6 secret-file
This is a file intended to be encrypted.

Wrote 1 line
Help Write Out Where Is Cut
Exit Read File Replace Paste Execute Justify
```

The terminal then shows the user navigating to the directory and creating the file:

```
(jenn@kali-linux)-[~]
$ cd /media/veracrypt1
cd: no such file or directory: /media/veracrypt1
(jenn@kali-linux)-[~]
$ cd /media/veracrypt27
(jenn@kali-linux)-[/media/veracrypt27]
$ nano secret-file
(jenn@kali-linux)-[/media/veracrypt27]
$ cat secret-file
This is a file intended to be encrypted.
(jenn@kali-linux)-[/media/veracrypt27]
$
```

METHODOLOGY:

To complete this lab, I followed the instructor-provided steps for installing and configuring VeraCrypt inside a Kali Linux virtual machine. I performed all technical actions directly within the lab4 VM, including downloading the correct VeraCrypt AppImage, generating the keyfile, creating the hidden volume, and mounting the encrypted container. All screenshots in the Appendix were captured from my own VM during the implementation process.

I used Microsoft Copilot as a support tool during the lab to clarify instructions, troubleshoot VirtualBox display and input issues, and help identify the correct sequence of steps when the interface was unclear or partially off-screen. Copilot also assisted in organizing and labeling the screenshots after the implementation was complete. All decisions about the configuration, passwords, keyfiles, and volume settings were made independently, and all actions inside the VM were performed manually.

To ensure accuracy, I cross-checked each step against the official VeraCrypt documentation and the lab instructions provided in Canvas. When the VM froze or the display became unusable, I used standard VirtualBox recovery techniques (e.g., toggling input capture, adjusting display settings, and restarting the display manager) to regain access without losing progress. After completing the implementation, I verified the encrypted volume by mounting it, creating a test file, and confirming that the file was stored inside the encrypted container.

This methodology reflects a combination of hands-on technical execution, reference to official documentation, and limited AI assistance for troubleshooting and documentation organization. All final work, analysis, and implementation decisions reflect my own understanding **and effort**.

REFERENCES:

Cimpanu, C. (2019). *Man who refused to decrypt hard drives still in prison after two years*. BleepingComputer. <https://www.bleepingcomputer.com/news/legal/man-who-refused-to-decrypt-hard-drives-still-in-prison-after-two-years/>

IDRIX. (n.d.). *Hidden volume*. <https://veracrypt.io/en/Hidden%20Volume.html>

IDRIX. (n.d.). *Plausible deniability*. <https://veracrypt.io/en/Plausible%20Deniability.html>

Sawant, A. (2025). *3_Lab VeraCrypt.docx* [Unpublished course document]. Northeastern University.

Sawant, A. (2025). *Module 3 – Lab: Implement VeraCrypt* [Course assignment instructions]. Northeastern University.